ZAINAB LORGAT

AUTHENTICATION TESTING

WHAT IS AUTHENTICATION TESTING?

- Systematic evaluation of how an app verifies user identity
- Tests logic, tokens, flows, and trust assumptions... not just passwords
- Covers: token issuance, MFA integrity, auth logic flaws, SSO trust chains
- OWASP OTG-AUTHN has 9 dedicated test categories
- Brute force & default creds exist but the interesting bugs live deeper

OWASP®

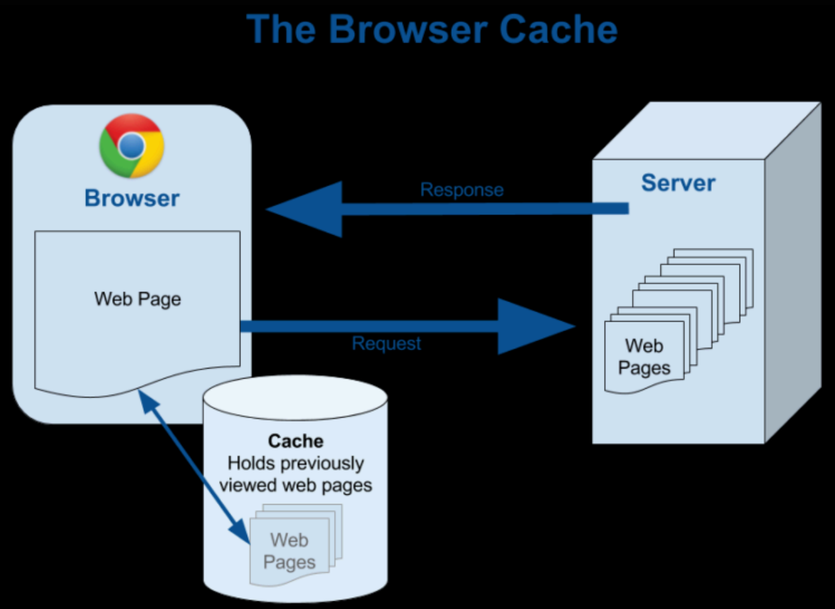
WHY IT MATTERS

- Authentication is the front door of every application
- If it fails, nothing else matters, firewalls, encryption, audit logs are all bypassed
- An attacker who passes auth IS the user, as far as the system is concerned
- It's not just about passwords... tokens, sessions, and MFA can all be the weak link
- The damage isn't just technical: reputation, trust, legal liability
- Most auth bugs aren't exotic, they're logic mistakes hiding in plain sight

TECHNIQUE 01: BROWSER CACHE WEAKNESSES

Does the browser store authenticated pages or credentials that can be retrieved without logging in?

- Back Button Attack: Log out → press back → authenticated page still visible
- Cached Credentials: Check if browser auto-saves credentials from non-HTTPS or misconfigured forms
- Developer Tools: Inspect cache storage, session storage, local storage for tokens or sensitive data post-logout
- Shared Device Risk: On a shared/public machine, cached authentication data, free access



TECHNIQUE 02: BYPASSING AUTHENTICATION SCHEMA

What it tests: Can you reach authenticated pages without ever logging in?

- Direct Page Request: **Navigate directly** to /dashboard, /admin, /account, does the app redirect you or let you in?
- Force Browsing: Guess or enumerate URLs that should require authentication
- SQL Injection on Login: **' OR '1'='1** in username/password field
- Modify Request Path: Change **POST /login** to **GET /login**
- Old/Backup Pages: /login_old, /admin2, /test

Tools: Burp Suite, OWASP ZAP, Nikto, manual browsing

What it tests: Can the credential recovery process be abused to take over an account?

- Host Header Injection: Swap Host header → reset link sent to attacker's server
- Token Predictability: Request 10–20 tokens rapidly → analyse for patterns
- Token Reuse
- No Current Password: Change password without confirming the existing one
- Username Manipulation: Tamper the username field in the reset request

Tools: Burp Suite (Intercept + Sequencer), manual analysis

TECHNIQUE 03: WEAK PASSWORD CHANGE OR RESET FUNCTIONALITIES

KEY TAKEAWAYS

1. Set Cache-Control headers... no-store, no-cache on all authenticated responses
2. Every protected page must verify session server-side on every single request
3. Password reset tokens must be: high entropy, single-use, short-lived
4. Never trust the Host header to build URLs

"Test authentication like someone who really wants in"

OWASP.ORG/WWW-PROJECT-WEB-SECURITY-TESTING-GUIDE PORTSWIGGER.NET/WEB-SECURITY OWASP.ORG/TOP10